

Fondation de SRPL pour un langage procédural relationnel strict

Document livré en Markdown, prêt à être enregistré tel quel en fichier `.md`.

Cadre et ambition

La naissance de SRPL est justifiée par une idée historique très simple : un langage relationnel utile n'est pas seulement un outil pour « écrire des requêtes », c'est une interface qui doit permettre à un humain d'exprimer une intention de données sans connaître les chemins physiques d'accès, l'implantation interne ni les structures de navigation du moteur. L'article fondateur de Edgar F. Codd ¹ pose déjà cette indépendance logique, et l'histoire officielle de IBM ² rappelle que cette vision a ensuite conduit à SQL, d'abord pensé comme SEQUEL, « Structured English Query Language », précisément pour rapprocher l'expression des requêtes d'un niveau plus naturel pour l'utilisateur. ³

Le deuxième fait déterminant est que SQL ne s'est pas imposé uniquement par sa syntaxe, mais par un triptyque : un compromis lisible pour l'humain, une cible formelle suffisamment riche pour les moteurs, et une normalisation progressive qui a permis un consensus industriel massif. L'historique de standardisation montre que SQL a été normalisé par ANSI puis ISO, qu'il a évolué par parties, et que le standard 2023 reste encore structuré en parties séparées, dont un cadre général, une fondation et une partie dédiée aux modules procéduraux persistants SQL/PSM. Autrement dit, même le standard SQL reconnaît déjà qu'un langage de données sérieux n'est pas monolithique. ⁴

C'est précisément là que SRPL a du sens : non pas comme « un SQL renommé », mais comme une réponse moderne à un besoin que SQL n'a jamais complètement résolu proprement, celui d'un langage transactionnel relationnel, strict, typé, lisible, borné et conçu dès le départ pour l'interprétation, la compilation, la production de plans et la réutilisation via plan cache. Les travaux récents sur les langages relationnels modernes et sur les représentations abstraites des requêtes montrent qu'il est désormais réaliste de séparer le cœur sémantique d'un langage, ses modalités humaines, et sa représentation machine. C'est exactement l'espace dans lequel SRPL peut être conçu proprement en 2026. ⁵

Ce que SQL a réussi et ce qu'il faut corriger

Il faut d'abord reconnaître ce que SQL a réellement apporté. SQL a rendu la manipulation relationnelle accessible à un large public, a donné une surface lisible pour la sélection, la restriction, les jointures et les agrégations, et a permis la traduction de requêtes de haut niveau en plans exécutables efficaces. Les moteurs actuels conservent cette force : les dialectes procéduraux de Microsoft ⁶, de Oracle Corporation ⁷, du PostgreSQL Global Development Group ⁸ et de IBM montrent tous qu'il existe une demande durable pour du code transactionnel côté serveur, proche des données, compilable, réutilisable et observable. ⁹

Mais SQL a aussi accumulé plusieurs défauts structurels que SRPL doit corriger au niveau du langage, pas seulement au niveau des bonnes pratiques. Le premier défaut est l'ambiguïté sémantique créée par l'algèbre des sacs. Dans les grands moteurs SQL, `SELECT ALL` est le comportement par défaut ; les

lignes dupliquées sont donc conservées sauf élimination explicite. PostgreSQL le documente explicitement, Db2 aussi, et les opérateurs `UNION`, `INTERSECT` et `EXCEPT` eux-mêmes balancent entre versions distinctes et versions `ALL`. Cela a rendu l'expression relationnelle plus performante dans certains cas historiques, mais beaucoup moins pure, moins prévisible et plus coûteuse à raisonner formellement. ¹⁰

Le deuxième défaut est la gestion de l'absence de valeur. SQL a normalisé `NULL` et la logique ternaire, alors même que les travaux de sémantique formelle montrent que cette troisième valeur ne procure pas de gain d'expressivité indispensable dans le cœur du langage. Le résultat pratique, lui, est connu : comportements difficiles à lire, propagation d'« unknown », et conventions implicites dans les filtres, les comparaisons et certaines agrégations. Pour SRPL, cela signifie qu'il faut refuser l'absence implicite et préférer une absence typée, explicite et vérifiable. ¹¹

Le troisième défaut est la faiblesse du noyau procédural historique. Les dialectes procéduraux actuels sont utiles mais restent souvent des extensions empilées au-dessus de SQL plutôt que de véritables langages conçus autour d'un modèle transactionnel strict. Un exemple frappant : en T-SQL, `BEGIN...END` groupe des instructions mais ne crée pas de portée lexicale ; une variable déclarée dans un bloc reste visible dans le batch parent. À l'inverse, PL/pgSQL autorise des variables `RECORD` sans structure prédéfinie, dont la forme dépend de l'affectation reçue à l'exécution. Ces choix sont pratiques pour l'auteur du dialecte, mais nuisibles pour la compilabilité forte et pour la raison formelle qu'ambitionne SRPL. ¹²

Le quatrième défaut est plus opérationnel : les extensions procédurales SQL sont énormément utilisées, mais elles restent souvent séquentielles, imbriquées et difficiles à optimiser. L'étude de Microsoft Research sur des milliers de procédures, UDFs et triggers montre une forte fréquence de `IF` imbriqués, des boucles présentes dans une part non négligeable des objets, et un constat global : la plupart des moteurs exécutent encore le procédural SQL de manière itérative et plutôt sérielle. Les travaux Froid puis Aggify sont importants ici parce qu'ils montrent tous deux la même direction : ouvrir le procédural, le ramener vers une représentation relationnelle plus explicite, puis laisser l'optimiseur le traiter comme un objet de plan, pas comme une boîte noire. ¹³

La conséquence est nette : SRPL doit conserver ce que SQL a rendu possible — consensus, optimisation, proximité avec les données, lisibilité en anglais, fonctions d'agrégation natives, transactions explicites — mais il doit refuser ce qui rend SQL difficile à verrouiller conceptuellement : sémantique multiensemble par défaut, `NULL` ambiant, formes procédurales insuffisamment typées, portée pas assez stricte, généricité trop souvent basée sur des chaînes dynamiques, et séparation trop faible entre la syntaxe de surface et la sémantique réelle. ¹⁴

Principes fondateurs de SRPL

Le premier principe de SRPL doit être **la phrase anglaise compréhensible**. SQL a gagné une partie de sa diffusion parce qu'il utilisait déjà des mots anglais simples et un ordre de lecture stable ; l'histoire de SEQUEL et les explications de IBM le montrent clairement. SRPL doit pousser cela plus loin : non pas en devenant du pseudo-langage naturel ambigu, mais en imposant des phrases normalisées, courtes, canoniques, toujours lisibles dans le même sens. La priorité doit être la compréhension immédiate de l'intention, y compris par une personne métier non programmeuse. ¹⁵

Le deuxième principe doit être **un cœur relationnel avec syntaxe object-like, sans basculer dans un modèle de persistance objet**. La bonne direction n'est pas l'identité objet, l'héritage persistant ou la surcharge implicite de comportements, mais des formes fermées, structurées et typées qui

ressemblent aux objets parce qu'elles sont plus faciles à lire : des enregistrements fermés, des membres nommés, des valeurs immuables par défaut, des états explicites. Les records de C# et les types structurés raffinés par narrowing en TypeScript montrent deux idées précieuses pour SRPL : la lisibilité des formes de données et la réduction contrôlée des cas possibles par analyse de flux. ¹⁶

Le troisième principe doit être **la sémantique d'ensemble par défaut**. Une relation SRPL n'est pas un sac. Si une multiplicité doit être conservée, elle doit être introduite explicitement comme un concept d'interface ou de compatibilité, jamais comme la sémantique implicite du noyau. Cela remet SRPL du côté du modèle relationnel originel et rend les preuves, la composition, la lecture des invariants et l'optimisation sémantique beaucoup plus stables. ¹⁷

Le quatrième principe doit être **l'absence typée, jamais NULL ambiant**. Le langage de surface doit utiliser une forme explicite telle que `optional`, `missing`, `present`, `absent`, ou une union discriminée équivalente. La logique des prédicats doit rester strictement booléenne dans le cœur du langage. Toute compatibilité avec des colonnes SQL nullable doit être repoussée à la frontière d'interopérabilité, avec conversion explicite vers un type optionnel et obligations de traitement par branche. ¹⁸

Le cinquième principe doit être **la cardinalité explicite dans la syntaxe**. SQL masque trop souvent l'intention cardinale réelle : l'auteur « espère » un singleton, mais écrit une requête qui peut retourner plusieurs lignes. SRPL doit forcer l'auteur à déclarer s'il attend `one`, `optional one`, `many` ou `nonempty many`. Cela n'est pas un détail cosmétique : c'est un contrat de compilation, de validation et de cache de plan. Cette recommandation prolonge logiquement le typage fort de C#, le narrowing de TypeScript et la nature typée de l'algèbre relationnelle. ¹⁹

Le sixième principe doit être **la portée lexicale stricte**. Si une variable ou une forme est déclarée dans un bloc, elle vit dans ce bloc et uniquement là. SRPL ne doit pas répéter la souplesse historique de T-SQL sur `BEGIN...END`, ni l'instabilité structurelle d'un `RECORD` réellement shape-shifting. Pour un langage compilable et optimisable, la forme d'un nom, sa portée et sa nature doivent être décidables statiquement. ²⁰

Le septième principe doit être **la transaction comme contrat, pas comme décoration**. Une procédure SRPL doit déclarer son mode transactionnel, son niveau d'isolation, son mode d'accès (`read only` / `read write`), ses collections lues et écrites, et idéalement ses invariants métier. Les SGBD majeurs documentent explicitement les effets du niveau d'isolation sur la visibilité des écritures, les anomalies et les garanties ; SRPL doit donc rendre ces choix visibles dans la signature de la procédure, au lieu de les laisser diffus dans la session ou dans des commandes annexes. ²¹

Le huitième principe doit être **la boucle bornée uniquement**. Les travaux empiriques sur le procédural SQL et les recherches d'optimisation de UDFs convergent : les boucles non structurées dégradent la lisibilité, l'optimisation et souvent la performance. SRPL doit donc interdire les formes ouvertes de `while` et `goto` dans son noyau, et ne permettre que des itérations sur des ensembles finis ou des répétitions à borne explicite. Le langage doit rendre la borne visible et exploitable par le compilateur. ²²

Le neuvième principe doit être **une séparation stricte entre syntaxe de surface et représentation sémantique**. Les travaux récents sur un Abstract Relational Query Language insistent sur un point capital : il faut distinguer le cœur relationnel, les modalités humaines et la représentation machine. Pour SRPL, cela signifie qu'il faut viser dès l'origine une syntaxe humaine anglaise, mais compiler

systématiquement vers un IR sémantique stable — un ALT ou équivalent — sur lequel s'appuieront validation, normalisation, optimisation, génération de plan et plan cache. ²³

Modèle sémantique et système de types

Le système de types SRPL doit être fermé, riche et orienté intention. Il doit distinguer au minimum les **types scalaires** (`Boolean`, `Int32`, `Int64`, `Decimal(p,s)`, `Text(n)`, `Date`, `Time`, `Timestamp`, `Uuid`, `Binary(n)`), les **types métier** dérivés (`CustomerId`, `OrderNumber`, `EmailAddress`), les **records** fermés, les **entities** relationnelles, les **types optionnels** et les **types de collection**. Les moteurs SQL modernes montrent déjà la diversité des catégories de types disponibles, mais SRPL doit les encapsuler dans une discipline beaucoup plus cohérente et bien plus prévisible que les dialectes historiques. ²⁴

Je recommande que SRPL différencie explicitement **value objects**, **records** et **entities**. Un value object est une valeur métier sans identité persistante indépendante ; un record est une forme structurée fermée ; une entity est une forme dont l'identité relationnelle est portée par une clé déclarée. Cette distinction s'aligne bien avec l'approche « data-centric » des records en C#, tout en évitant l'ambiguïté objet/rangée. Elle permet aussi de clarifier les jointures : on joint des relations d'entities, on projette des records, on agrège vers des records de synthèse, et on retourne soit une cardinalité scalaire, soit une collection typée. ²⁵

Le cœur de SRPL doit ensuite être **cardinalement typé**. Une expression ne retourne pas seulement un type ; elle retourne un type et une cardinalité. Par exemple, `one Customer`, `optional one Customer`, `many Order`, `nonempty many InvoiceLine`. Cette information doit être portée jusque dans les mutations, les agrégations et les vérifications d'invariants. Elle remplace une vaste zone d'ambiguïté pratique du SQL et prépare naturellement la génération de diagnostics de compilation, d'alertes de cardinalité et de plans plus stables. ²⁶

Pour l'absence de valeur, SRPL doit adopter un **type optionnel discriminé**. La bonne sémantique n'est pas « la valeur manque et la logique devient ternaire », mais « la forme du résultat dit s'il y a ou non une valeur, et le code doit traiter le cas ». Le modèle de narrowing de TypeScript et le pattern matching de C# sont particulièrement utiles ici : ils montrent qu'un langage peut rester très lisible tout en exigeant que l'auteur traite explicitement les branches de présence, d'absence ou d'état. En conséquence, un `optional one Customer` doit être traité par une construction dédiée, jamais par un `=` ou un `IS NULL` hérité. ²⁷

Concernant les collections, SRPL doit distinguer **relation**, **projection**, **group**, **index** et **sequence**. La `relation` est un ensemble non ordonné d'éléments typés. La `projection` est un record dérivé. Le `group` est une partition déterminée par une clé. L'`index` est une vue clé-valeur utile pour l'accès logique dans une procédure. La `sequence`, si elle existe, ne doit pas être un concept relationnel de base mais une vue ordonnée explicite, utile pour l'interface, les exports ou certaines itérations bornées. C'est une correction directe aux surprises d'ordre et de `TOP/LIMIT` qu'on rencontre en SQL. ²⁸

Les agrégations doivent être **natives**, **typées** et **explicites sur le cas vide**. Les grands moteurs SQL offrent déjà un large éventail de fonctions d'agrégation — `COUNT`, `AVG`, `MIN`, `MAX`, `SUM`, fonctions statistiques, ordered-set aggregates — mais leurs conventions de comportement sur ensemble vide, ordre de traitement, ou interaction avec les `NULL` ne sont pas toujours évidentes. SRPL doit donc rendre explicite le comportement recherché, par exemple `sum of Amount else 0`, `average of Score else absent`, `list of Name ordered by Name`. Ainsi, les conventions cessent d'être implicites et deviennent partie intégrante de l'intention. ²⁹

Enfin, je recommande que SRPL impose des **prédicats lisibles et nommés**. À la place d'un `IN (...)` hérité comme simple reliquat syntaxique, la bonne direction est une forme anglaise stable telle que `is any of`, `belongs to`, `matches`, `satisfies`, `exists in`, `none in`. Pour les jointures, `match ... by ...` est sémantiquement plus explicite qu'un simple `JOIN` laissé à des conventions implicites. Le vocabulaire doit traduire une volonté : trouver, regrouper, comparer, garantir, créer, modifier, retirer, retourner. C'est ce champ verbal qui fera de SRPL un vrai langage d'intention transactionnelle et non un assemblage d'opérateurs. ³⁰

Syntaxe proposée pour SRPL

La syntaxe SRPL doit être construite de sorte qu'une procédure se lise comme un contrat, puis comme une suite d'intentions. Je recommande une structure stable en six parties : **signature**, **capacités**, **convention sémantique**, **déclarations**, **mutations**, **retour**. Les verbes centraux devraient être limités et canoniques : `define`, `take`, `let`, `find`, `collect`, `match`, `group`, `compute`, `ensure`, `create`, `change`, `remove`, `return`, `fail`. Cette réduction du vocabulaire est importante : elle favorise l'apprentissage, la lisibilité et la normalisation du parser vers un IR unique. ³¹

Voici une forme de base cohérente avec vos objectifs :

```
transaction procedure CloseExpiredReservations
takes
  Today as Date
reads
  Reservations as set of Reservation
writes
  Reservations as set of Reservation
with
  isolation as serializable,
  access mode as read write,
  semantics as set,
  missing values as explicit
do
  let expiredReservations be many Reservation
    from Reservations
    where Reservation.ExpiresAt < Today
      and Reservation.Status is ReservationStatus.Open;

  ensure no Reservation in expiredReservations
    satisfies Reservation.IsLocked is true;

  change each Reservation in expiredReservations
    set
      Reservation.Status to ReservationStatus.Closed,
      Reservation.ClosedAt to Today;

  return count of expiredReservations;
end procedure
```

Cette forme est volontairement plus stricte que T-SQL, PL/SQL ou PL/pgSQL. Elle rend visibles la transaction, l'ensemble des collections touchées, la sémantique d'ensemble, la gestion explicite de l'absence et la cardinalité attendue. Elle est également plus adaptée à une compilation en plan stable : les accès, les effets et la nature du résultat sont déjà présents dans la source. Cette orientation est cohérente avec les travaux sur SQL/PSM, sur les représentations abstraites des requêtes, sur LINQ en tant que requête de premier ordre du langage, et sur les techniques qui traduisent le procédural en représentation relationnelle optimisable. ³²

Pour les recherches avec prédicats, les regroupements, les `IN`, les jointures et les agrégats, je recommande une syntaxe SRPL de ce style :

```
transaction procedure BuildCustomerBalanceSummary
takes
  CustomerIds as many CustomerId
reads
  Customers as set of Customer,
  Invoices as set of Invoice
returns
  many CustomerBalanceSummary
do
  let selectedCustomers be many Customer
    from Customers
    where Customer.Id is any of CustomerIds;

  let summaries be many CustomerBalanceSummary
    from selectedCustomers as Customer
    match Invoices as Invoice
      by Customer.Id = Invoice.CustomerId
    group by Customer.Id, Customer.Name
    compute
      CustomerId      as Customer.Id,
      CustomerName    as Customer.Name,
      InvoiceCount     as count of Invoice,
      TotalOpenAmount as sum of Invoice.OpenAmount else 0,
      HighestBalance  as max of Invoice.OpenAmount else 0;

  return summaries;
end procedure
```

L'intérêt de cette forme est triple. D'abord, elle lit l'intention dans l'ordre humain : on part d'une source, on restreint, on associe, on regroupe, on calcule. Ensuite, elle enlève plusieurs implicites SQL classiques : pas de `SELECT *`, pas de colonnes positionnelles, pas de bag semantics cachée, pas de comportement indéfini sur ensemble vide. Enfin, elle favorise une canonicalisation très forte : `from`, `match`, `group`, `compute` deviennent des nœuds sémantiques stables, faciles à descendre vers un ALT puis vers une algèbre relationnelle d'exécution. ³³

Pour l'absence explicite, SRPL doit éviter absolument que l'auteur se repose sur une logique ternaire implicite. Je recommande une écriture dédiée :

```

let maybeCustomer be optional one Customer
  from Customers
  where Customer.Id = CustomerId;

when maybeCustomer has value as Customer do
  return Customer;
otherwise
  fail with CustomerNotFound;
end when

```

Cette construction force un traitement franc du cas d'absence. Elle est plus verbeuse qu'un `WHERE Id = @Id` suivi d'une supposition implicite de singleton, mais cette verbosité est productive : elle déplace une ambiguïté d'exécution vers une intention explicite de l'auteur. C'est exactement le type de « strictness » que SRPL doit assumer. ³⁴

Pour les boucles, ma recommandation est d'autoriser uniquement deux formes : **l'itération sur collection finie** et **la répétition bornée**. Par exemple :

```

for each Line in RequestedLines do
  ensure Line.Quantity > 0;
end for

repeat 3 times do
  collect diagnostics;
end repeat

```

En revanche, je déconseille un `while` généraliste dans le noyau de SRPL. Les moteurs SQL historiques l'acceptent, mais les études d'usage et les travaux d'optimisation montrent que la boucle libre tire vite le langage vers une programmation séquentielle opaque pour le compilateur. Si SRPL veut être un vrai langage transactionnel déclaratif, il doit faire de la borne un concept syntaxique, pas une convention informelle. ³⁵

Compilation, exécution et plan cache

Si SRPL veut devenir un langage crédible pour les SGBDRT, son architecture d'exécution doit être pensée dès maintenant. Je recommande une chaîne de compilation en cinq étages : **parse**, **binding typé**, **ALT/IR sémantique**, **plan relationnel transactionnel**, **exécutable cachable**. Cette architecture s'appuie sur deux constats issus des sources : d'une part, LINQ a montré la valeur d'une requête devenue une construction du langage pouvant être traduite via des expression trees ; d'autre part, les travaux ARQL/ARC rappellent qu'une bonne représentation abstraite doit refléter la sémantique et non la syntaxe de surface. ³⁶

À l'étage **binding typé**, SRPL doit résoudre tous les noms, tous les types métier, toutes les cardinalités et tous les droits d'effet. Aucune ambiguïté de nom, aucune source implicite, aucune colonne anonyme ne doit subsister. C'est aussi à cet étatge que les conventions du langage doivent être fixées : sémantique d'ensemble, politique sur l'absence, comportement des agrégats sur ensemble vide, mode transactionnel et contraintes de portée. Le papier ARC est particulièrement utile ici parce qu'il montre qu'un certain nombre de comportements traditionnellement confondus avec « la langue elle-même »

relèvent en réalité de conventions orthogonales ; SRPL gagnerait énormément à expliciter ces conventions dans l'environnement de compilation. 37

À l'étage **planification**, SRPL peut tirer directement les leçons de Froid et d'Aggify. Le bon objectif n'est pas de tolérer un procédural opaque puis d'essayer de l'exécuter honorablement ; le bon objectif est de réduire toute procédure SRPL à une forme suffisamment relationnelle pour que l'optimiseur voie le plus possible du programme. Froid montre qu'une traduction systématique d'un corps impératif en expressions algébriques relationnelles ouvre la porte à l'optimisation à coût, aux plans ensemblistes et au parallélisme ; Aggify montre qu'une partie des boucles sur curseurs peut être éliminée au profit d'agréats plus natifs. SRPL doit être conçu pour rendre ces transformations naturelles, pas héroïques. 38

Pour le **plan cache**, les SGBD majeurs fournissent déjà les preuves pratiques de faisabilité. SQL Server compile une procédure au premier appel, conserve le plan si possible et le réutilise ensuite ; PostgreSQL met en cache ou réutilise les plans des instructions PL/pgSQL optimisables ; Oracle compile les unités PL/SQL, peut les stocker sous forme compilée et supporte même la compilation native. SRPL doit s'inspirer de ce modèle mais avec de meilleurs identifiants sémantiques : le cache doit être indexé par signature de procédure, empreinte de schéma, conventions sémantiques, types de paramètres, mode transactionnel et éventuellement forme normalisée de l'ALT. 39

En pratique, cela signifie qu'un plan SRPL ne devrait pas être « le plan d'un texte ». Il devrait être **le plan d'une intention liée**. C'est un point crucial si vous visez à terme un interpréteur capable ensuite de produire du code compilable, puis du code-plan et du plan cache robustes. Tant que le cache dépend trop d'une syntaxe de surface permissive, il reste fragile. Dès que le cache dépend d'une forme sémantique canonique, le langage devient une vraie interface de compilation. 40

Positionnement, gouvernance et feuille de route

La décision la plus importante à prendre est peut-être conceptuelle : **SRPL doit remplacer SQL côté transactionnel, mais il ne doit pas essayer de devenir à lui seul tout ce que SQL a historiquement empilé**. Le standard SQL lui-même sépare déjà cadre, fondation et modules procéduraux. Je recommande donc de traiter SRPL comme le noyau procédural et transactionnel strict d'une famille de langages, avec le même système de types, le même modèle sémantique et le même IR, plutôt que comme un « super dialecte » chargé de tout faire d'un coup. 41

Autrement dit, si l'ambition de long terme est bien de « remplacer SQL entièrement », la stratégie la plus saine n'est probablement pas un seul bloc appelé SRPL chargé à la fois des schémas, des requêtes ad hoc, des procédures, des migrations, des privilèges et de la méta-programmation. La stratégie la plus propre serait une **famille SRxL** partageant un même cœur strict. SRPL en serait la partie procédurale transactionnelle. Une future sous-partie de définition de schéma pourrait être distincte pour conserver à SRPL sa netteté conceptuelle. Cette manière de faire serait bien plus cohérente avec l'héritage ISO que de reconstruire un nouveau monolithe. 42

Pour une feuille de route 2026, je recommande trois paliers. Le premier palier est un **noyau minimal viable** : types fermés, cardinalités explicites, `from/where/match/group/compute`, `create/change/remove`, transactions déclarées, boucle bornée, agrégats et absence explicite. Le deuxième palier est l'**outilage** : binder, ALT, diagnostics de cardinalité, diagnostics de portée, visualisation de plan et hash sémantique de cache. Le troisième palier est l'**interopérabilité** : import de schémas SQL existants, surface de compatibilité nullable vers `optional`, et traduction contrôlée de certains objets SQL vers SRPL — mais jamais l'injection de SQL dynamique brut dans le noyau. Cette

progression est cohérente avec les enseignements combinés de LINQ, des recherches ARQL, des optimisations Froid/Aggify et des pratiques actuelles de plan caching. ⁴³

Enfin, un dernier point de gouvernance me paraît essentiel : **SRPL doit définir très tôt ce qui est interdit**. C'est un langage strict ; sa valeur viendra autant de ses capacités que de ses refus explicites. Je recommanderais donc d'interdire dès la première version du noyau : `SELECT *`, les noms implicites ou positionnels, les blocs sans portée, les valeurs nulles implicites, les unions de formes incompatibles, les boucles non bornées, les conversions silencieuses, le SQL dynamique textuel dans le noyau, et toute sémantique multiensemble implicite. Un langage vraiment moderne pour 2026 ne doit pas seulement ajouter des fonctionnalités ; il doit couper proprement les héritages qui empêchent la raison formelle, la compilation stable et la compréhension humaine. ⁴⁴

Questions ouvertes et limites

Le corpus ci-dessus est une fondation robuste, mais il reste plusieurs questions de design qui doivent être tranchées avant de figer SRPL.

D'abord, il faut décider si **SRPL reste strictement procédural transactionnel** ou s'il devient le noyau visible d'une famille plus large incluant plus tard la définition de schéma. Les sources ISO poussent clairement vers une séparation par parties, et cette direction me paraît préférable. ⁴²

Ensuite, il faut définir précisément la **frontière d'interopérabilité** avec l'existant : comment importer le nullable SQL, comment gérer les objets hérités qui portent déjà des doublons fonctionnels, et jusqu'où accepter des compatibilités temporaires avec des idiomes de dialectes historiques sans contaminer le noyau strict. Les divergences documentées entre T-SQL, PL/SQL, PL/pgSQL et SQL PL montrent que cette frontière doit être pensée comme une couche de migration, jamais comme une concession faite au cœur du langage. ⁴⁵

Enfin, plusieurs champs restent à spécifier dans une future version normative : le traitement des requêtes récursives, les agrégats utilisateurs, la temporalité, la sécurité déclarative, l'observabilité native, la représentation des diagnostics, et le statut éventuel des types avancés modernes comme `json`, `vector` ou certains types spatiaux. Les moteurs actuels les exposent déjà, mais SRPL devra décider s'ils appartiennent au noyau, à des bibliothèques standard, ou à des extensions de plateforme. ⁴⁶

Le point clé, malgré ces questions ouvertes, est déjà net : SRPL ne doit pas chercher à être « encore un SQL ». Il doit être un **langage procédural relationnel strict**, à intention visible, sémantique d'ensemble, absence explicite, portée verrouillée, cardinalité déclarée, compilation native vers un IR sémantique, et exécution pensée dès l'origine pour le plan et le plan cache. Si vous tenez cette ligne, SRPL a une identité propre, défendable théoriquement, exploitable industriellement, et réellement modernisatrice pour 2026. ⁴⁷

¹ ⁸ ¹⁰ ²⁸ ³³ ⁴⁴ <https://www.postgresql.org/docs/current/sql-select.html>
<https://www.postgresql.org/docs/current/sql-select.html>

² ⁵ ²³ ³⁷ ⁴⁰ <https://www.vldb.org/cidrdb/papers/2026/p27-gatterbauer.pdf>
<https://www.vldb.org/cidrdb/papers/2026/p27-gatterbauer.pdf>

³ ¹⁷ ⁴⁷ https://kataix.umag.cl/~jaguila/Databases/Paper_Codd.pdf
https://kataix.umag.cl/~jaguila/Databases/Paper_Codd.pdf

4 14 https://sigmodrecord.org/publications/sigmodRecord/1806/pdfs/08_Industry_Michels.pdf
https://sigmodrecord.org/publications/sigmodRecord/1806/pdfs/08_Industry_Michels.pdf

6 11 18 https://www.vldb.org/pvldb/vol11/p27-guagliardo.pdf?utm_source=consensus
https://www.vldb.org/pvldb/vol11/p27-guagliardo.pdf?utm_source=consensus

7 41 42 <https://www.iso.org/standard/76583.html>
<https://www.iso.org/standard/76583.html>

9 45 <https://docs.oracle.com/en/database/oracle/oracle-database/23/Inpls/overview.html>
<https://docs.oracle.com/en/database/oracle/oracle-database/23/Inpls/overview.html>

12 20 <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/begin-end-transact-sql?view=sql-server-ver17>
<https://learn.microsoft.com/en-us/sql/t-sql/language-elements/begin-end-transact-sql?view=sql-server-ver17>

13 22 <https://15799.courses.cs.cmu.edu/spring2025/papers/19-udfs/p1378-ramachandra.pdf>
<https://15799.courses.cs.cmu.edu/spring2025/papers/19-udfs/p1378-ramachandra.pdf>

15 30 <https://www.ibm.com/think/topics/structured-query-language>
<https://www.ibm.com/think/topics/structured-query-language>

16 25 <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/records>
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/records>

19 <https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview>
<https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview>

21 <https://learn.microsoft.com/en-us/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?view=sql-server-ver17>
<https://learn.microsoft.com/en-us/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?view=sql-server-ver17>

24 46 <https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver17>
<https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver17>

26 <https://dtai-static.cs.kuleuven.be/ilp-mlg-srl/USBStick/papers/ILP09-4.pdf>
<https://dtai-static.cs.kuleuven.be/ilp-mlg-srl/USBStick/papers/ILP09-4.pdf>

27 <https://www.typescriptlang.org/docs/handbook/2/narrowing.html>
<https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

29 <https://learn.microsoft.com/en-us/sql/t-sql/functions/aggregate-functions-transact-sql?view=sql-server-ver17>
<https://learn.microsoft.com/en-us/sql/t-sql/functions/aggregate-functions-transact-sql?view=sql-server-ver17>

31 36 43 <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>
<https://learn.microsoft.com/en-us/dotnet/csharp/linq/>

32 <https://www.iso.org/standard/76585.html>
<https://www.iso.org/standard/76585.html>

34 <https://arxiv.org/abs/2012.13198>
<https://arxiv.org/abs/2012.13198>

35 <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/control-of-flow?view=sql-server-ver17>
<https://learn.microsoft.com/en-us/sql/t-sql/language-elements/control-of-flow?view=sql-server-ver17>

38 <https://15799.courses.cs.cmu.edu/spring2025/papers/19-udfs/p432-ramachandra.pdf>
<https://15799.courses.cs.cmu.edu/spring2025/papers/19-udfs/p432-ramachandra.pdf>

³⁹ <https://learn.microsoft.com/en-us/sql/relational-databases/query-processing-architecture-guide?view=sql-server-ver17>
<https://learn.microsoft.com/en-us/sql/relational-databases/query-processing-architecture-guide?view=sql-server-ver17>